

SDN-Scale: Comparação entre Árvores AVL e Red-Black para Otimização de Roteamento

Gabriel Guarnieri Fraga e Sousa, João Inácio Rodrigues Torres de Souza,
Guilherme Augusto de Moraes Araújo

iCEV - Instituto de Ensino Superior
Teresina - PI - Brasil

{gabriel.guarnieri, joao.souza, guilherme.araujo}@somosicev.com

Abstract. This paper compares AVL and Red-Black trees in a simulated edge-routing scenario for packet rules. The implementation evaluates insertion, search, deletion of 20% of the rules, height, rotations, and structural invariants. The same fixed seed was used for both structures, making the tests uniform. Results indicate that AVL produced lower height and better search behavior in the evaluated read-intensive scenario, while Red-Black showed slightly fewer rotations in some measurements.

Resumo. Este artigo compara árvores AVL e Red-Black em um cenário simulado de roteamento de borda para regras de pacotes. A implementação avalia inserção, busca, remoção de 20% das regras, altura, rotações e invariantes estruturais. A mesma seed fixa foi usada nas duas estruturas, mantendo uniformidade nos testes. Os resultados indicam que a AVL obteve menor altura e melhor comportamento de busca no cenário avaliado, enquanto a Red-Black apresentou quantidade ligeiramente menor de rotações em algumas medições.

1. Introdução e Fundamentação

Em sistemas de roteamento e balanceamento de carga, a eficiência da busca por regras influencia diretamente a latência percebida pelo usuário e a capacidade do sistema de operar sob alto volume de requisições. Em um cenário de redes definidas por software, regras de firewall e de encaminhamento podem ser inseridas, consultadas e removidas com frequência, exigindo estruturas de dados que mantenham desempenho previsível mesmo com entradas sequenciais.

Uma árvore binária de busca comum pode se degenerar quando recebe chaves ordenadas, aproximando seu comportamento de uma lista encadeada. Para evitar esse problema, este trabalho compara duas árvores auto-balanceadas: AVL e Red-Black. Ambas possuem operações com custo assintótico $O(\log n)$, mas adotam estratégias diferentes de balanceamento.

A árvore AVL mantém uma invariante mais rígida: para cada nó, a diferença entre as alturas das subárvores esquerda e direita deve permanecer entre -1 e 1. Por isso, tende a produzir árvores mais baixas e buscas mais rápidas. A Red-Black, por sua vez, usa coloração de nós e regras de altura negra, aceitando um balanceamento mais flexível em troca de menor custo médio de manutenção estrutural.

2. Metodologia

A solução foi desenvolvida em Java, sem dependências externas obrigatórias para execução do núcleo do projeto. Foram implementadas duas estruturas: AVLRouterTree e RedBlackRouterTree, ambas manipulando objetos PacketRule, compostos por identificador, IP de origem, IP de destino e prioridade.

Para manter a comparação justa, as regras foram geradas pela classe RuleGenerator usando a seed fixa 20240520. Os mesmos objetos gerados foram inseridos nas duas árvores, garantindo que os resultados não fossem influenciados por diferenças na massa de dados.

Os volumes testados foram 1.000, 10.000, 50.000 e 100.000 entradas. Para cada volume, foram medidas três operações principais: inserção de todas as regras, busca de todas as regras e remoção de 20% das regras. Os tempos foram coletados em nanossegundos por meio de System.nanoTime().

Além dos tempos, também foram registradas a altura final de cada árvore, a quantidade total de rotações e o tamanho final após remoção. Os resultados foram exportados para o arquivo results/benchmark_results.csv.

3. Desenvolvimento

3.1. Implementação da AVL

A AVL foi implementada com nós contendo referência à regra armazenada, ponteiros para filho esquerdo e direito e a altura do nó. A cada inserção ou remoção, a altura é recalculada e o fator de balanceamento é verificado. Quando o fator sai do intervalo permitido, a árvore executa rotações simples ou duplas para restaurar a invariante.

As operações implementadas foram insert, search, delete, rotateLeft, rotateRight e rebalance. Também foi adicionada uma contagem de rotações para permitir análise quantitativa do custo de manutenção da estrutura.

3.2. Implementação da Red-Black

A Red-Black foi implementada com nós coloridos em vermelho ou preto, ponteiros para filhos e ponteiro para o pai. A implementação utiliza um nó sentinela NIL preto para representar folhas externas, o que simplifica a lógica de rotação e correção após inserções e remoções.

Foram implementados os métodos insert, fixInsert, delete, fixDelete, rotateLeft e rotateRight. A raiz é mantida sempre preta, nós vermelhos não podem possuir filhos vermelhos, e todos os caminhos da raiz até as folhas NIL devem apresentar a mesma altura negra.

4. Validação das Invariantes

O Integrante 3 adicionou testes de validação para conferir se as estruturas permaneceram corretas após inserções e após a remoção de 20% das regras. Essa etapa foi necessária porque tempos de execução só são relevantes quando a estrutura resultante permanece válida.

Na AVL, os testes verificam a propriedade de árvore binária de busca, a altura armazenada em cada nó e o fator de balanceamento no intervalo entre -1 e 1. Na Red-

Black, os testes verificam se a raiz é preta, se nós vermelhos não possuem filhos vermelhos, se a altura negra é igual em todos os caminhos, se os ponteiros de pai estão corretos e se a propriedade de árvore binária de busca foi preservada.

5. Resultados Comparativos

Tabela 1. Tempos medidos nas operações principais

Estrutura	Entradas	Inserção (ns)	Busca (ns)	Remoção 20% (ns)
AVL	1.000	3.511.500	227.200	525.700
Red-Black	1.000	1.269.300	516.300	194.100
AVL	10.000	3.257.500	2.272.200	1.088.400
Red-Black	10.000	3.851.900	2.412.100	3.241.300
AVL	50.000	11.709.000	4.644.200	1.437.500
Red-Black	50.000	12.227.400	4.546.400	8.300.400
AVL	100.000	17.042.800	7.105.200	6.309.900
Red-Black	100.000	18.251.800	8.180.500	6.046.100

Tabela 2. Métricas estruturais após as operações

Estrutura	Entradas	Tamanho final	Rotações	Altura
AVL	1.000	800	1.088	10
Red-Black	1.000	800	1.083	16
AVL	10.000	8.000	10.984	14
Red-Black	10.000	8.000	10.976	23
AVL	50.000	40.000	54.981	16
Red-Black	50.000	40.000	54.970	28
AVL	100.000	80.000	109.980	17
Red-Black	100.000	80.000	109.968	30

Os resultados indicam que a AVL apresentou menor altura final em todos os volumes. No maior teste, com 100.000 entradas, a AVL terminou com altura 17, enquanto a Red-Black terminou com altura 30. Essa diferença ajuda a explicar o desempenho melhor da AVL nas buscas.

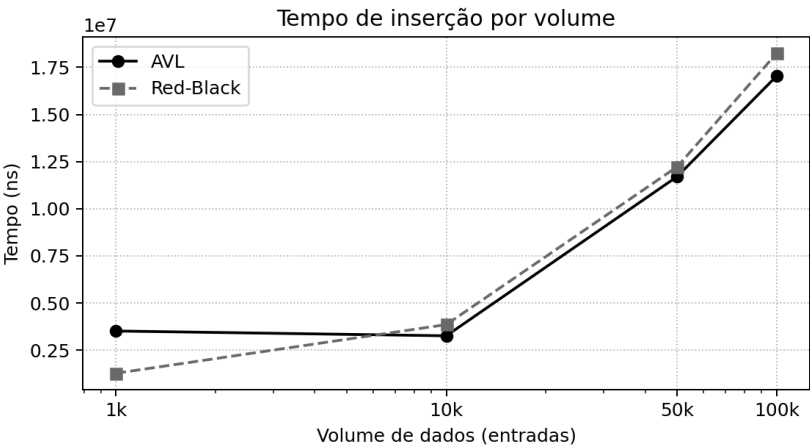


Figura 1. Comparação do tempo de inserção por volume de dados

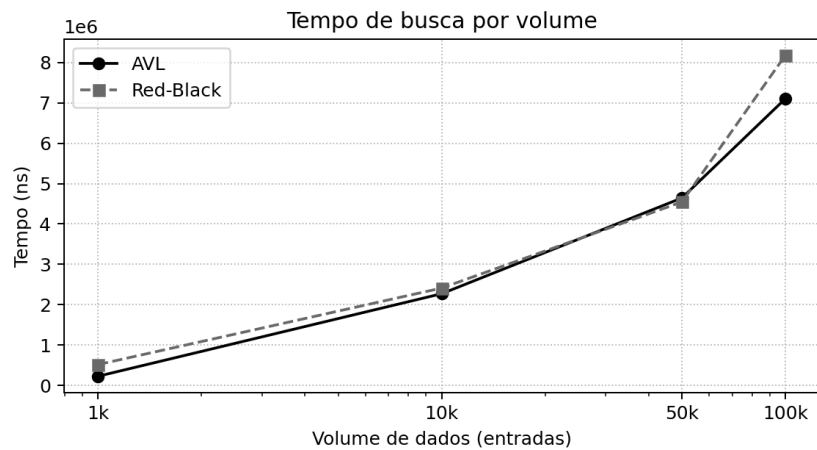


Figura 2. Comparação do tempo de busca por volume de dados

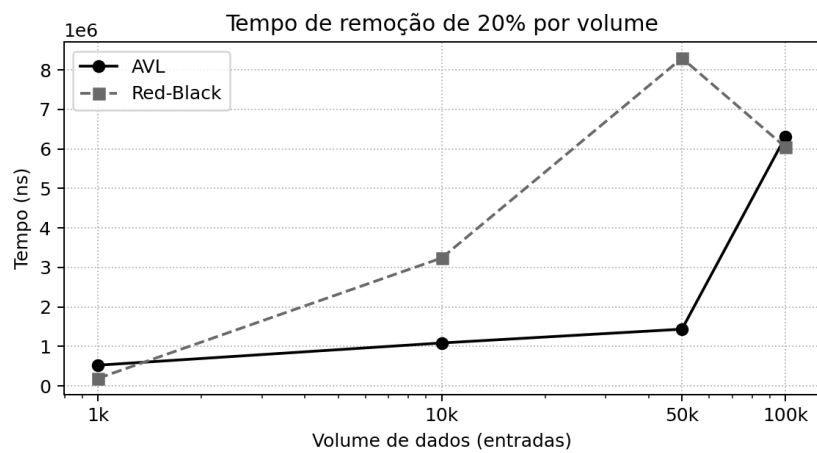


Figura 3. Comparação do tempo de remoção de 20% por volume de dados

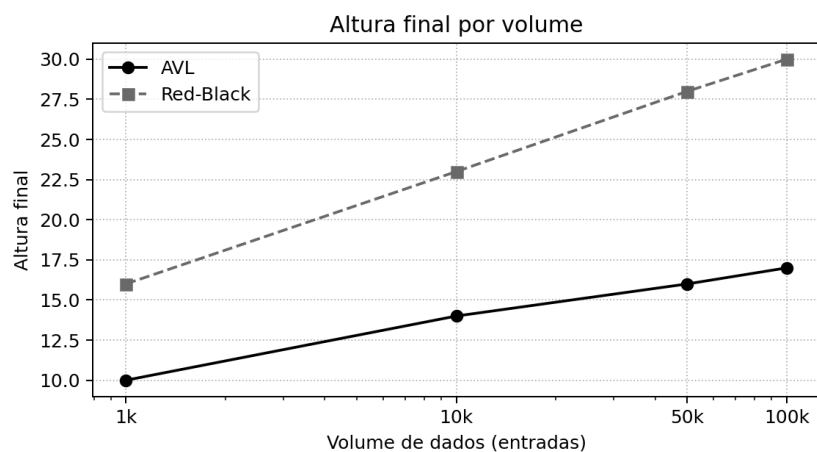


Figura 4. Comparação da altura final por volume de dados

6. Discussão: Custo de Rotação e Trade-offs

As duas árvores apresentaram quantidades de rotações muito próximas. A Red-Black teve uma quantidade ligeiramente menor de rotações em todos os volumes

medidos, o que está de acordo com sua proposta de balanceamento menos rígido. Entretanto, essa diferença foi pequena quando comparada ao impacto da altura final da árvore nas buscas.

A AVL, por manter equilíbrio mais rigoroso, gerou árvores mais baixas. Em um cenário em que a busca por regras ocorre com maior frequência do que alterações estruturais, essa característica é vantajosa. A Red-Black continua sendo uma boa alternativa quando a aplicação possui grande volatilidade, com muitas inserções e remoções, mas neste experimento a AVL se mostrou mais adequada ao objetivo principal.

A remoção de 20% das regras funcionou como teste de estresse para o rebalanceamento. Em ambas as estruturas, o tamanho final ficou correto em todos os volumes, e os validadores de invariantes confirmaram que as árvores não foram corrompidas após a exclusão.

7. Post-Mortem Técnico

O problema investigado foi a possibilidade de latência excessiva em um mecanismo de roteamento quando regras são inseridas sequencialmente. Em uma árvore binária comum, esse padrão poderia causar forte desbalanceamento. Para mitigar esse risco, foram implementadas e avaliadas duas estruturas auto-balanceadas.

A análise mostrou que a AVL reduziu a altura da estrutura e apresentou melhor comportamento nas buscas, principalmente no cenário de 100.000 entradas. A Red-Black apresentou menor número de rotações, mas sua altura final foi maior, o que prejudicou a busca em alguns volumes.

A decisão técnica recomendada para a diretoria do projeto é utilizar AVL como motor principal de consulta de regras no cenário avaliado. O motivo é que o sistema simulado é sensível à latência de busca, e a AVL apresentou menor profundidade, mantendo as invariantes após inserção e remoção.

Como ações futuras, recomenda-se repetir os benchmarks em múltiplas rodadas, calcular médias e desvios, testar entradas aleatórias além das ordenadas e avaliar o comportamento em cenários com proporção maior de remoções e atualizações.

8. Conclusão

Este trabalho implementou e comparou árvores AVL e Red-Black aplicadas ao gerenciamento de regras PacketRule em um cenário simulado de roteamento de borda. Foram avaliados tempos de inserção, busca e remoção de 20%, além de altura final e quantidade de rotações.

Os resultados indicaram que a AVL é a estrutura mais adequada para o cenário proposto, especialmente pelo menor custo de busca associado à menor altura final. A Red-Black apresentou comportamento correto e competitivo, mas sua maior altura a torna menos atrativa para um cenário read-intensive.

O projeto também demonstrou práticas de integração em equipe, com branches, pull requests, revisão de código e validação de invariantes antes da integração final. Esse fluxo reforça a importância de combinar implementação, medição, auditoria e comunicação técnica.

Referências

Cormen, T. H., Leiserson, C. E., Rivest, R. L. and Stein, C. (2009). Introduction to Algorithms. 3rd edition. MIT Press.

Goodrich, M. T., Tamassia, R. and Goldwasser, M. H. (2014). Data Structures and Algorithms in Java. 6th edition. Wiley.

Oracle. Java Platform Documentation. System.nanoTime() method. Disponível na documentação oficial da linguagem Java.

Sociedade Brasileira de Computação. Modelo para artigos e resumos de conferências da SBC.